

M1.1: Tokenizer & Lexer

Theory: Incremental slice tokenization and Span physical positioning.

Details: `rustc_lexer` uses hand-crafted DFAs to split utf-8 source code into `TokenKind` streams while keeping track of Span pointers (32-bit compact offsets) for syntax trace-backs.

Trade-off: To avoid allocation overheads, tokens do not duplicate string literals but contain reference spans, requiring the entire source file buffer to remain in memory.

M1.4: Type Inference & HM Algorithm

Theory: Type variables generation, Unification, and type equation solving.

Details: Employs Hindley-Milner type inference extended with trait constraints. It constructs type placeholders during checks, building equivalence matrices resolved via a Unification Table.

Trade-off: Complex nested generics or long fluent API call chains degrade unification performance; use local type annotations to speed up solving.

M2.1: HIR to MIR Lowering

Theory: Constructing Control Flow Graphs (CFGs) and flat basic block representations.

Details: Flattens HIR into a Control Flow Graph (MIR). Complex nested matches and logical operations are lowered into Basic Blocks consisting of Assignments and Terminators.

Trade-off: Simplifies later stages but creates many tiny basic blocks, requiring subsequent compiler cleanup passes.

M2.2: MIR Dataflow Analysis

Theory: CFG tracking, Def-Use chains, and variable liveness estimation.

Details: Performs iterative liveness analyses on CFG nodes. Tracks variables through Gen/Kill sets at basic block entry/exit points to determine ownership states.

Trade-off: Algorithms require scanning the control flow graph until convergence, with performance depending on CFG complexity and loops.

M1.2: Macro Expansion & Name Resolution

Theory: Nested macro expansion, name binding, and syntax symbol indexing.

Details: `rustc_resolve` walks the syntax tree twice. First it expands macros and resolves imports, then it binds names across three independent namespaces (Value, Type, Macro).

Trade-off: Heavy macro nesting and wildcard imports (`'use a:*`) bloat name resolution graphs and slow down initial compilation phases.

M1.5: Trait Coherence & Co-existence

Theory: Enforces Specialization rules and Orphan Rules to prevent traits collision.

Details: Coherence verifies that no two trait implementations overlap. The Orphan Rule mandates that either the trait or the target struct belongs to the current compiling crate.

Trade-off: Orphan rules trade coding flexibility for global coherence, preventing compiler crashes caused by overlapping third-party implementations.

M2.3: NLL Region Inference

Theory: Flexible lifetime region tracking at specific CFG evaluation points.

Details: Non-Lexical Lifetimes (NLL) analyzes reference usage locations instead of checking lexical scopes, building subtyping constraint systems on CFG points.

Trade-off: Drastically increases computational overhead for region solvers, especially in massive generator blocks or async states.

M1.3: AST to HIR Lowering

Theory: Desugaring syntax trees into High-Level Intermediate Representations (HIR).

Details: Lowers AST syntax nodes to HIR while desugaring loops (`'for'` to `'loop'/'match'`), bindings, and analyzing closure escape captures.

Trade-off: Desugaring exposes extra compiler variables, swelling memory footprints. `rustc` uses arena-based page allocation to speed up HIR generation.

M1.6: Method Resolution & Deref Coercions

Theory: Deref chain traversal and method signature candidate search.

Details: Resolves `'x.foo()'` calls by checking `'T'`, `'&T'`, `'&mut T'`, then applying Deref coercions (e.g. `'Box<T> -> T'`) recursively to seek matching candidate methods.

Trade-off: Deeply nested dereferences (e.g. `'&&&&T'`) lengthen candidate search chains, increasing method lookup latency.

M2.4: Borrow Checker Validation

Theory: Enforces read/write lock constraints across inferred NLL regions.

Details: Tracks borrows at compile time. If an active borrow region overlaps with a write access, or outlives the owner container, a compile-time error is raised.

Trade-off: Guarantees memory safety without run-time barriers, but introduces steep learning curves for programmers.

M2.5: Drop Elaboration

Theory: Injects drop flags and execution cleanup branches in destructors.

Details: Since variables may be conditionally moved, rustc injects implicit stack boolean flags (Drop Flags) and cleanup execution paths on panic paths.

Trade-off: Adds tiny runtime stack markers, though static analysis removes redundant flags whenever lifetimes are known.

M3.2: Dynamic Dispatch & VTables

Theory: Fat pointer layouts and virtual method table dispatch.

Details: '&dyn Trait' holds two pointers: one pointing to struct data and the other to a read-only VTable containing drop glues, alignment metadata, and function pointers.

Trade-off: Keeps binary sizes compact, but indirect jumps through VTables bypass CPU branch predictors and prevent inline optimizations.

M3.5: Linker Bridge & LTO

Theory: Coordinates cross-crate LTO and native system linkers.

Details: Invokes platform linkers (lld, link.exe) at the final stage. Link-Time Optimization (LTO) aggregates LLVM Bitcode across crates for global optimizations.

Trade-off: LTO drastically increases link times and host RAM usage; keep it disabled during local developer loops.

M4.1: Safe/Unsafe Boundaries

Theory: Isolates unchecked operations from standard borrow-checked blocks.

Details: Inside 'unsafe' blocks, developers can dereference raw pointers, call unsafe methods, mutate static variables, or implement unsafe traits.

Trade-off: Shifts safety verification responsibility to developers; mistakes lead directly to undefined behavior.

M2.6: MIR Optimizations

Theory: Local optimizations including constant folding, inlining, and copy propagation.

Details: rustc_mir_transform optimizes MIR graphs before LLVM translation, utilizing precise borrow data to safely execute inline expansion and redundant moves cleanup.

Trade-off: MIR passes only perform high-yield optimizations, leaving heavy hardware-specific instruction optimization to LLVM.

M3.3: Memory Layout & Auto Traits

Theory: Physical struct alignment reordering and Send/Sync static verification.

Details: Reorders struct fields to minimize padding bytes unless marked '#[repr(C)]'. Synthesizes Auto Traits (Send/Sync) by checking all structural fields.

Trade-off: Breaks classic C memory sequence assumptions; developers must specify C representation layouts for ABI foreign function interfaces.

M3.6: Incremental Compilation & Query System

Theory: Breaks compilation into cached query dependencies.

Details: Tracks tasks as atomic queries (e.g., 'type_of(Id)'). File modifications update the Dep-Graph, invalidating only affected queries while reusing cached calculations.

Trade-off: Maintaining dependency graphs consumes extra compiler memory; bad modular design can invalidate large portions of the graph.

M3.1: Generic Monomorphization

Theory: Compiles generic functions into type-specific concrete machine blocks.

Details: Translating 'foo:<i32>' and 'foo:<String>' duplicates the template MIR code, specializing instructions for each target type configuration.

Trade-off: Generates raw optimized assembly with no jump overheads, but causes binary bloat and scales compilation times with generic complexity.

M3.4: LLVM IR Translation

Theory: Translates MIR nodes and types into LLVM SSA instruction sets.

Details: Maps MIR basic blocks and types to LLVM C++ APIs, translating assignments and terminators into LLVM IR (e.g. StructType, alloca, br).

Trade-off: High inter-process communication overhead between rustc and LLVM; cg_clif (Cranelift) can be used to speed up local testing builds.

M3.7: Stack Unwinding & Panic Runtime

Theory: Landing pads and destructor execution during panic stack unwinding.

Details: If panic unwind is enabled, rustc generates Landing Pads, using C++ ABI unwinding to walk up the stack and run destructors until caught.

Trade-off: Landing pads bloat binaries. Switch to 'panic = "abort"' in embedded or low-space targets to call core abort.

M4.2: Undefined Behavior & Miri

Theory: Stacked Borrows alias tracking and runtime UB analysis.

Details: Miri runs MIR in a virtual interpreter, maintaining a Stacked Borrows model to track borrow permissions and detect violation patterns.

Trade-off: Miri execution is extremely slow, making it suitable only for unit testing rather than production execution.

M4.3: CTFE Const Evaluation Engine

Theory: Compile-Time Function Evaluation (CTFE) in-compiler VM.

Details: `rustc_const_eval` interprets MIR structures at compile time, resolving arrays, sizes, and const evaluations without runtime code.

Trade-off: The const VM lacks OS system capabilities (no net, no files) and limits instruction counts to avoid compiler hangs.

M4.4: Thread Safety Send/Sync Constraint

Theory: Static compilation checks preventing concurrent data races.

Details: Send/Sync traits are automatically derived. If a type holds non-Send fields (e.g., Rc), the compiler flags threads spawning attempts as type mismatches.

Trade-off: Prevents concurrency data races, but limits threading flexibility and requires explicit Arc/Mutex abstractions.

M4.5: Macro Hygiene & Proc-Macro Bridge

Theory: Avoids name pollution and facilitates proc-macro compilation.

Details: Macros use hygienic scopes to isolate variables. Proc-macros compile as shared libraries, communicating with rustc via a TokenStream compiler bridge.

Trade-off: Proc-macro expansion relies on dynamic library loading and IPC, increasing compiler launch overheads.

M5.1: Diagnostics Engine & Suggestions

Theory: Span tracking and structured compiler error diagnostics.

Details: `rustc_errors` parses Span metadata to display context highlighted code blocks and generate diff suggestions for automated fixes.

Trade-off: Diagnostics add minor rendering times to build pipelines, though the developer productivity gains are massive.

M5.2: Query Cache & Dependency Graph

Theory: DepGraph propagation and selective dirty query invalidation.

Details: DepGraph tracks inputs/outputs. File changes mark source nodes, propagating bottom-up to mark affected query values as dirty.

Trade-off: Maintaining query dependency relationships consumes significant compiler memory during builds.

M5.3: Unstable Feature Gates

Theory: Door locks controlling nightly feature gates.

Details: Non-stabilized features require '#![feature(...)]' markers. rustc checks compiler channels and aborts if unstable flags are run on stable.

Trade-off: Restricts experimental APIs to nightly, keeping stable code backward-compatible.

M5.4: Crate Metadata Serialization

Theory: Encodes public symbols and MIR templates to rmeta files.

Details: `rustc_metadata` serializes public signatures, trait tables, and inlineable MIR to rmeta files, allowing external projects to load dependencies quickly.

Trade-off: Parsing and decoding large dependency metadata maps adds minor I/O latency.

Zone T1: rustc Core APIs

```
rustc_driver::run_compiler(),

rustc_interface::interface::run_compiler(),

rustc_middle::mir::Body, rustc_borrowck::borrowck(),

rustc_codegen_ssa::traits::CodegenMethods,

rustc_trait_selection::traits::fulfill(), rustc_resolve::Resolver
```

Zone T2: Compilation & Execution Errors

```
borrowck_lifetime_conflict, trait_selection_overflow,

monomorphization_recursion_limit,

const_eval_undefined_behavior, meta-
data_serialization_mismatch, macro_hygiene_expansion_error
```